

ĐẶC TẢ KỸ THUẬT VÀ KIẾN TRÚC PHẦN MỀM (TECH SPEC)

Tên dự án: CatLife

Môn học: Lập trình Hướng đối tượng (OOP) - ĐHBKHN

Công nghệ sử dụng: Java 17, Framework libGDX (Core, Box2D, Freetype, Scene2D)

Mô hình vận hành: Game Loop, Lập trình hướng sự kiện (Event-Driven), Thời gian thực (Real-time Simulation).

PHẦN 1: TỔNG QUAN CẤU TRÚC DỰ ÁN (PACKAGE STRUCTURE)

Mã nguồn được chia tách theo nguyên tắc Separation of Concerns (SoC) nhằm đảm bảo code không bị dính chùm (decoupled), dễ bảo trì và mở rộng.

hust.hedspi.oop.catlife

- core/ # Entry point (CatLifeGame), cấu hình chung
- managers/ # Các Singleton Controller (GameManager, TimeManager...)
- entities/ # Các đối tượng vật lý (Player, NPC, Đồ vật)
- components/ # Các hệ thống phụ trợ (Trạng thái, Kỹ năng)
- screens/ # Các màn hình chính (Menu, Map, Ending)
- minigames/ # Chứa logic tách biệt của các minigame (Strategy Pattern)
- utils/ # Hằng số (Constants), tiện ích tính toán
- debug/ # Hệ thống Debug Kit (Cửa sổ F12)

PHẦN 2: HỆ THỐNG QUẢN LÝ LỖI (CORE MANAGERS)

Sử dụng Singleton Pattern để đảm bảo chỉ có 1 phiên bản tồn tại, giúp truy xuất dữ liệu toàn cục an toàn.

0. GameManager (Design Pattern: Singleton)

- Vị trí vai trò: Nếu TimeManager chỉ lo đếm giờ, StoryManager chỉ lo cốt truyện, thì GameManager là Người điều phối tổng (Coordinator). Nó nắm giữ trạng thái toàn cục của phiên chơi hiện tại và làm cầu nối để các hệ thống khác giao tiếp với nhau mà không bị phụ thuộc vòng tròn (circular dependency).

- Chức năng và Trách nhiệm chính (What it does):

- Quản lý Trạng thái Game (Game State Machine): Lưu trữ vòng đời của game (PLAYING, PAUSED, IN_MINIGAME, GAME_OVER). Ví dụ: Khi người chơi bật Cửa sổ F12 hoặc ấn nút Esc, GameManager sẽ đổi state sang PAUSED, từ đó báo cho TimeManager dừng đếm giờ và Player dừng di chuyển.
- Nắm giữ tham chiếu Nhân vật chính (Player Reference): Mọi class, quái vật, NPC hay Minigame nếu muốn biết vị trí của chú Mèo (Player) hoặc muốn trừ máu Mèo, chúng không cần phải tìm kiếm vòng vo mà chỉ cần gọi GameManager.getInstance().getPlayer().

3. Khởi tạo & Reset Phiên chơi (Session Management): Khi người chơi bấm "New Game" ở Main Menu, GameManager sẽ gọi hàm initNewGame(). Hàm này chịu trách nhiệm: Xóa data cũ của StoryManager, đưa TimeManager về Day 1, sinh ra một object Cat (Stray hoặc House) mới ở đúng tọa độ xuất phát.
4. Điều hướng sự kiện Sinh tử (Life & Death Cycle): Nó liên tục lắng nghe máu của Mèo. Nếu máu Mèo về 0, GameManager sẽ đóng băng game và lập tức gọi StoryManager để ép xuất ra Bad Ending hoặc Màn hình Game Over.

1. ScreenManager: Quản lý lịch sử màn hình bằng cấu trúc dữ liệu Stack<Screen>.

Giúp game dễ dàng chuyển từ PlayScreen -> MinigameScreen -> PlayScreen (Pop stack).

2. TimeManager: Trái tim của cơ chế Free-time (Stardew Valley style).

- Quy đổi: 1 giây đời thực = 2 phút trong game.
- Cập nhật Hour, Minute, DayOfWeek liên tục mỗi khung hình (frame).
- Tự động bắn sự kiện (Notify) khi đổi khung giờ (Sáng/Tối) để đổi màu Map.

3. StoryManager: Quản lý các cờ sự kiện (Story Flags).

Sử dụng HashMap<EventFlag, Boolean> để lưu trữ (Ví dụ: HAS_WIFE: true, IS_CASTRATED: false).

4. SoundManager: Quản lý BGM (Nhạc nền) và SFX (Hiệu ứng), hỗ trợ fade in/fade out âm thanh.

PHẦN 3: HỆ THỐNG THỰC THỂ VÀ OOP ĐA HÌNH (ENTITIES)

Đội Dev phải tuân thủ chặt chẽ Tính Kế thừa (Inheritance) và Đa hình (Polymorphism).

3.1. Hệ thống Interfaces

- IInteractable: Cấp cho NPC, thùng rác, cửa ra vào. Chứa hàm onInteract(Cat player).
- IDamageable: Cấp cho đối tượng có thanh máu. Chứa hàm takeDamage(float amount).

3.2. Cây Kế thừa Nhân vật (Entity Hierarchy)

- Entity (Abstract Class): Lớp cha chứa x, y, width, height, hitbox, và hàm trừu tượng update(float dt), render(SpriteBatch batch).
- Cat (Abstract Class extends Entity):
 - Tính Đóng gói: Giữ private các chỉ số (HP, Hunger, Energy, Karma) và cung cấp Getter/Setter an toàn.
 - Chứa hàm abstract applyPassiveSkill().
- StrayCat & HouseCat (Concrete Classes):
 - Kế thừa lớp Cat.
 - Tính Đa hình: Ghi đè (Override) hàm applyPassiveSkill(). StrayCat có nội tại tăng sức mạnh khi ít máu; HouseCat có nội tại hồi phục nhanh khi ở trong nhà.

PHẦN 4: CHIẾN LƯỢC ÁP DỤNG DESIGN PATTERNS

Đây là những Pattern bắt buộc phải code để tối ưu hệ thống:

1. State Pattern (Máy trạng thái của Player)

- Mục đích: Khử các vòng lặp if-else phức tạp trong hàm di chuyển của mèo.
- Cách làm: Tạo interface ICatState với các hàm enter(), update(), exit(). Các class cụ thể: IdleState, RunState, SleepState. Mèo giữ biến currentState và ủy quyền logic cho nó.

2. Observer Pattern (Đồng bộ UI và Dữ liệu)

- Mục đích: Tránh việc màn hình HUD (Thanh Máu, Thời gian) phải hỏi data liên tục (gây lag).
- Cách làm: Cat và TimeManager là các Subject. Màn hình HUD là Observer. Chỉ khi Mèo bị trừ máu hoặc Thời gian nhảy sang giờ mới, nó mới gửi tin nhắn (Notify) ép UI vẽ lại.

3. Strategy Pattern (Hệ thống Minigame)

- Mục đích: Quản lý 10 minigame khác nhau mà không cần tạo 10 Screen rườm rà.
- Cách làm: Tạo interface IMinigameStrategy với hàm start(), render(), isFinished(). Truyền class Minigame cụ thể (Ví dụ: RhythmHiphopGame) vào trong 1 MinigameScreen chung để chạy thuật toán tương ứng.

4. Chain of Responsibility / Priority Evaluation (Hệ thống Kết Cục)

- Mục đích: Đánh giá Ending khi hết game mà không dùng if-else lồng nhau.
- Cách làm: Mỗi Ending là 1 Class kế thừa EndingCondition, mang một mức độ ưu tiên (Priority) và hàm isSatisfied(). Cuối game, StoryManager duyệt qua danh sách Ending đã sắp xếp, cái nào true trước sẽ được kích hoạt.

PHẦN 5: CƠ CHẾ SỰ KIỆN THEO THỜI GIAN (TIME-GATED SYSTEM)

Với thời gian trôi tự do, các NPC và khu vực tương tác hoạt động theo nguyên lý "Kích hoạt bằng điều kiện":

1. TimeCondition (Bộ Lọc Thời Gian):

- Mỗi Nhiệm vụ (Quest) / Sự kiện (Event) chứa một object TimeCondition (quy định Ngày hợp lệ, Giờ bắt đầu, Giờ kết thúc).

2. Trigger Zone (Vùng Va Chạm Ẩn):

- Sử dụng TiledMap (.tmx), tạo các hình chữ nhật ẩn (Hitbox) tại các khu vực sự kiện (VD: Hẻm tối).
- Khi tọa độ Mèo giao nhau (overlap) với Hitbox này -> Game gọi kiểm tra TimeCondition. Nếu đúng giờ, hiện nút [E] để người chơi kích hoạt Minigame hoặc Story.

3. Hệ thống Báo mộng (Foreshadowing):

- Khi thao tác [Ngủ] để qua ngày được kích hoạt, dựa vào biến Karma (Nghịệp) hiện tại, màn hình đen sẽ in ra 1 dòng Text (từ file cấu hình Excel) báo trước về Ending mà người chơi đang hướng tới.

PHẦN 6: BỘ CÔNG CỤ NHÀ PHÁT TRIỂN - DEBUG KIT (Phím F12)

Do tính chất game thời gian thực và cốt truyện phân nhánh, Team Dev sẽ xây dựng một giao diện Debug Menu (sử dụng thư viện Scene2D hoặc VisUI) nổi trên cùng màn hình.

Các tính năng bắt buộc của Debug Kit:

- Tab Thời gian: Nút Pause/Play, Nút tua nhanh (+1h, +6h, Next Day).
- Tab Player: Slider kéo thả để thay đổi lập tức HP, Hunger, Energy. Nút Kill (Ép chết).
- Tab Story: Danh sách Checkbox bật/tắt ép buộc các cờ sự kiện (Tích vào "Có Vợ", "Là Đại Ca").
- Tab Quick-Play: Các nút bấm rẽ nhánh trực tiếp vào test từng Minigame (Bỏ qua việc đi bộ trên Map) và Test Ending UI.

PHẦN 7: QUY TẮC VIẾT CODE DÀNH CHO TEAM DEV (CODING CONVENTIONS)

1. **Cấm Hardcode (Magic Numbers):** Toàn bộ kích thước, tốc độ, text thoại phải đưa vào file Constants.java hoặc file JSON/XML bên ngoài.
2. **Fail-Fast với Asset:** Tuyệt đối không dùng code chờ file ảnh từ team Design. Sử dụng ShapeRenderer (Vẽ cục vuông màu đỏ làm nhân vật, màu xanh làm quái) để code logic va chạm và trạng thái trước.
3. **Clean Code:**
 - Tên Class: PascalCase.
 - Tên Biến/Hàm: camelCase.
 - Tên Hằng số: UPPER_SNAKE_CASE.
 - Mọi logic thay đổi thông số phải đi qua hàm (Ví dụ: cat.decreaseEnergy(10) thay vì cat.energy -= 10).